

Proyecto 1: Diseño Lógico y FPGA

Calculadora de 4 bits en complemento a dos

Arquitectura de Computadores
Universidad de los Andes
Semestre 2026-2

Integrantes: _____

Septiembre de 2026

Este informe cubre los ítems 1 a 3 de la entrega: diseño general, tablas de verdad/Karnaugh/expresiones booleanas, e implementación mediante compuertas. El testbench, la simulación en GTKWave y el funcionamiento en la FPGA se muestran en la evaluación presencial. Todo el diseño está en `src/` del repositorio; este documento sigue el mismo orden y los mismos nombres de módulo que el código.

Índice

1	Diseño general y arquitectura de la calculadora	3
1.1	Especificación	3
1.2	Decisiones de diseño	3
1.3	Arquitectura	4
1.4	Los dos módulos de nivel superior	4
1.5	Interfaz de usuario y máquina de estados	4
1.6	Interfaz física y polaridad	5
1.7	Verificación	5
2	Tablas de verdad, mapas de Karnaugh y expresiones booleanas	7
2.1	Multiplexor 2:1 (<code>mux2</code>)	7
2.2	Sumador completo de 1 bit (<code>full_adder</code>)	7
2.3	Sumador de 4 bits (<code>adder4</code>)	7
2.4	Acondicionador de resta	8
2.5	Decodificador de operación (<code>op_decoder</code>)	8
2.6	Intercambio de operandos y barrel shifters	9
2.7	Selector de operación: contador módulo 6 (<code>op_selector</code>)	9
2.8	Contador módulo 16 (<code>counter4</code>)	10
2.9	Máquina de estados de control (<code>fsm_control</code>)	11
2.10	Signo y valor absoluto (<code>abs4</code>)	11
2.11	Decodificador de 7 segmentos (<code>seg7_decoder</code>)	11
3	Implementación de las operaciones mediante compuertas	14
3.1	Celdas básicas	14
3.2	Sumador de 4 bits	14

3.3	Decodificador de operación	14
3.4	ALU	15
3.5	Barrel shifter	15
3.6	Contador y máquina de estados	15
3.7	Valor absoluto y 7 segmentos	16
3.8	Conteo de compuertas por módulo	16

1 Diseño general y arquitectura de la calculadora

1.1 Especificación

La calculadora opera con números de 4 bits en complemento a dos (rango -8 a $+7$). Se mantuvo la codificación de operaciones del enunciado:

Código	Operación	Resultado	Observación
000	Reinicio	$R = 0000$	Ignora A y B
001	Suma	$R = A + B$	Módulo 16
010	Resta	$R = A - B$	Complemento a dos, módulo 16
011	Resta inversa	$R = B - A$	Complemento a dos, módulo 16
100	Shift left	$R = A \ll B[1:0]$	Lógico, rellena con ceros
101	Shift right	$R = A \gg B[1:0]$	Lógico, rellena con ceros
110, 111	inválido	$R = 0000$	No alcanzables desde los botones

Tabla 1: Codificación de operaciones.

Ningún archivo de `src/` usa `+` `-` `<` `>` `<=` `>=` `<<` `>>` `?:` `if` `case`: toda la lógica combinacional está en primitivas de compuerta (`and`, `or`, `not`, `xor`, `nand`, `nor`, `xnor`, `buf`). Lo único secuencial son los registros `always @(posedge clk) q <= d;`, con `d` calculado también con compuertas.

1.2 Decisiones de diseño

1. **Aritmética módulo 16, sin detectar overflow.** El enunciado pide conservar los 4 bits menos significativos, así que el acarreo de salida del sumador se descarta directamente.
2. **Resta con el mismo sumador.** $A - B = A + \overline{B} + 1$. Una sola señal `sub` invierte B bit a bit (XOR) y entra como acarreo inicial del sumador. Suma, resta y resta inversa comparten `adder4`.
3. **Resta inversa por intercambio de operandos.** En vez de un segundo restador, ocho `mux2` intercambian A y B cuando `swap`=1 antes de restar.
4. **Desplazamientos lógicos**, controlados solo por $B[1:0]$ como pide el enunciado; $B[3:2]$ se ignora.
5. **Decodificador one-hot + enmascarado.** `op_decoder` entrega seis líneas one-hot; la ALU las usa como máscara AND/OR a la salida. Con un código inválido (110, 111) las seis quedan en 0 y la salida es 0000: comportamiento seguro sin necesitar lógica extra.
6. **Signo y magnitud en el display.** Izquierdo: guion si el valor es negativo. Derecho: magnitud en hexadecimal. Como $|V| \leq 8$, los códigos 9–15 quedan disponibles como don't-care en el decodificador de 7 segmentos.
7. **FSM de 2 bits.** Codificando los cuatro estados como contador binario, el próximo estado es $s_1 \oplus s_0$ y $\overline{s_0}$: dos compuertas.
8. **Debounce por muestreo.** 3 muestras iguales seguidas (a 1,31 ms cada una) para aceptar un cambio, más detección de flanco para convertir el nivel en un pulso de un ciclo.
9. **Reset de encendido.** Los cuatro botones ya están asignados por el enunciado; `por` mantiene el reset activo 4 ciclos al partir para que la simulación no arranque en X.

1.3 Arquitectura

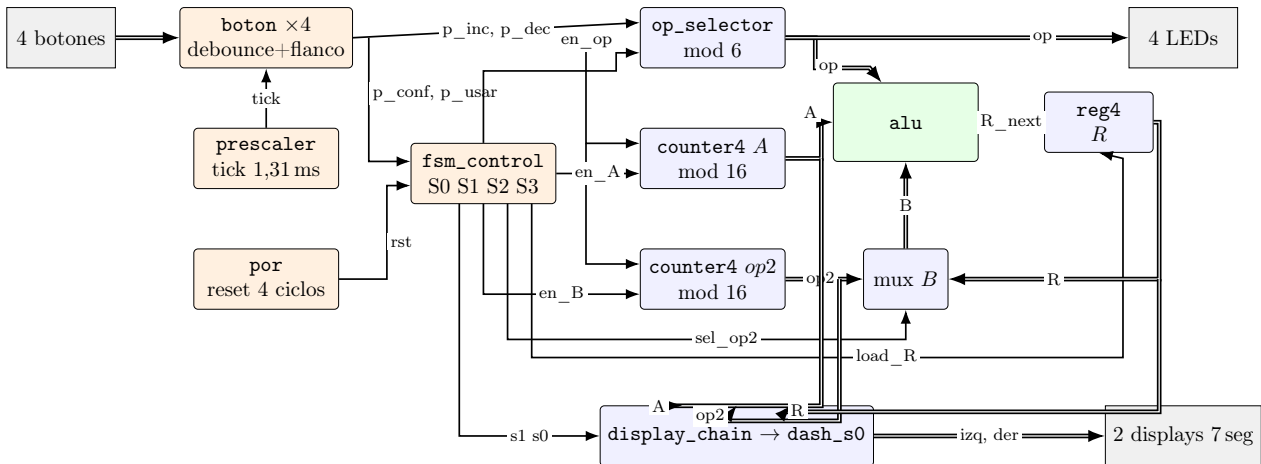


Figura 1: Diagrama de bloques (módulo `top`). Naranja: control e interfaz de botones. Azul: camino de datos. Verde: ALU combinacional.

`op_selector` no tiene entrada de habilitación propia, así que `top` filtra sus pulsos de entrada con `en_op` (dos `and`, `go1/go2`) antes de conectarlos: sin ese filtro, subir en cualquier otro estado también cambiaría la operación.

1.4 Los dos módulos de nivel superior

Hay dos módulos que comparten el mismo núcleo (`alu` + `reg4/reg4_ini`):

	<code>top</code>	<code>calculadora_4bits</code>
Entradas	4 botones físicos	<code>codigo</code> , <code>op1</code> , <code>op2_ext</code> , <code>sel_op2</code> , ejecutar directos
Control	FSM de 4 estados + debounce	una sola señal <code>ejecutar</code>
Salida	LEDs + 2 displays de 7 segmentos	resultado de 4 bits, sin decodificar

Tabla 2: `top` es la calculadora en la placa. `calculadora_4bits` es el envoltorio con la interfaz que usará el testbench de la evaluación (mismo `codigo` de 3 bits, mismo `op1/op2_ext` de 4 bits y el mismo selector de segundo operando de 1 bit que pide el enunciado). No reimplementa nada: instancia la misma `alu`.

`calculadora_4bits` usa `reg4_ini` en vez de `reg4`: es el mismo registro pero sin puerto de reset (la interfaz de la evaluación no lo tiene) y con valor inicial 0000 en la declaración del flip-flop en su lugar.

1.5 Interfaz de usuario y máquina de estados

Botón	Puerto	Función
Superior izquierdo	<code>i_Switch_1</code>	Incrementar
Inferior izquierdo	<code>i_Switch_2</code>	Disminuir
Superior derecho	<code>i_Switch_3</code>	Confirmar / avanzar de estado
Inferior derecho	<code>i_Switch_4</code>	Usar el resultado anterior como segundo operando

Tabla 3: Asignación de botones (Nandland Go Board).

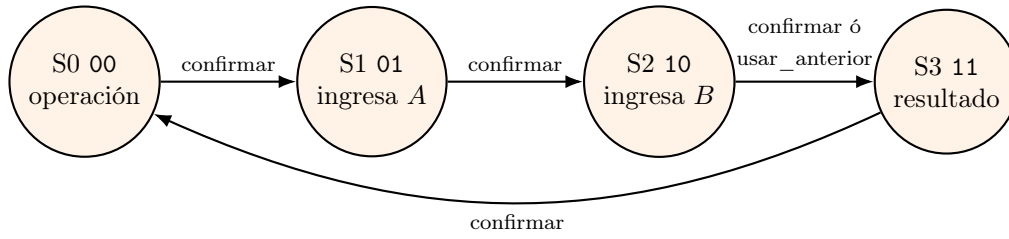


Figura 2: Máquina de estados de control (*fsm_control*).

$s_1 s_0$	Estado	Habilita	Botones izquierdos	Displays
00	S0	en_op	cambian el código de operación	guion en ambos
01	S1	en_A	cambian A	signo + magnitud de A
10	S2	en_B	cambian $op2$	signo + magnitud de $op2$
11	S3	en_R	sin efecto	signo + magnitud de R

Tabla 4: Comportamiento por estado. $op[2]$ es el LED 1 (bit más significativo).

En S2, si se presiona el botón inferior derecho, *sel_op2* (combinacional: *usar_anterior* AND *en_B*) hace que el mux de B tome R en vez de $op2$, y *load_R* carga el resultado en el mismo ciclo. La realimentación $R \rightarrow \text{mux} \rightarrow \text{ALU} \rightarrow \text{reg4} \rightarrow R$ no es un lazo combinacional porque el flip-flop de *reg4* corta el camino. Los operandos A y $op2$ no se reinician entre rondas; para poner el resultado en 0 se usa la operación 000.

1.6 Interfaz física y polaridad

- **Reloj:** oscilador de 25 MHz de la placa.
- **Botones y LEDs:** activos en alto. Entran por cuatro *buf* en *top.v*; si la polaridad resultara ser la contraria, se cambian esos cuatro *buf* por *not* y no hay que tocar nada más.
- **Displays de 7 segmentos (5261BG):** ánodo común, activos en *bajo*, verificado directamente en la placa. Toda la lógica interna sigue en lógica positiva; las 14 salidas de segmento pasan por un *not* final. Es De Morgan aplicado en la salida y no obliga a rehacer ningún mapa de Karnaugh.
- **Formato de bus:** $\text{seg}[6:0] = \{a, b, c, d, e, f, g\}$.

1.7 Verificación

Testbench	Casos	Resultado
<i>adder4_tb</i>	512 (barrido exhaustivo, $16 \times 16 \times 2$)	Sin errores
<i>shifter_tb</i>	64 por dirección (exhaustivo)	Sin errores
<i>op_selector_tb</i>	19 (subida, bajada, retención, prioridad, reset)	Sin errores
<i>alu_tb</i>	2048 (exhaustivo, $8 \times 16 \times 16$)	Sin errores
<i>display_tb</i>	107 (<i>abs4</i> , <i>seg7_decoder</i> , mux de fuente)	Sin errores
<i>debounce_tb</i>	11 (prescaler, rebote, glitch, flanco)	Sin errores
<i>control_tb</i>	79 (contadores, registro, FSM, one-hot)	Sin errores
<i>calculadora_4bits_tb</i>	17 (interfaz de la evaluación)	Sin errores
<i>top_tb</i>	33 (punta a punta, pulsaciones reales)	Sin errores
<i>demo_tb</i> , <i>alu_wave_tb</i>	editables, para la evaluación presencial	—

Tabla 5: Cobertura de simulación (Icarus Verilog). *alu_tb* verifica además que el decodificador se mantenga one-hot y que 110/111 dejen las seis salidas en 0.

Los once testbenches propios tampoco usan `if` ni `case`: para contar errores se usa que (`obtenido != esperado`) entrega 1 ó 0 y se acumula directo, y el veredicto final se imprime indexando un arreglo de strings con esa bandera. Los operadores aritméticos sólo aparecen ahí, para calcular el valor de referencia contra el que se compara el diseño de compuertas.

Síntesis (yosys 0.68 + nextpnr-ice40, iCE40 HX1K VQ100): 174 celdas lógicas de 1280 (13%), 56 flip-flops, 23 pines de 72 (31%). La netlist son 166 SB_LUT4 + 56 SB_DFF. Frecuencia máxima 122,31 MHz tras el ruteo, contra los 25 MHz del oscilador de la placa. Sin latches inferidos ni lazos combinacionales.

2 Tablas de verdad, mapas de Karnaugh y expresiones booleanas

Convención: $\bar{x}$ es la negación, la yuxtaposición es AND, $+$ es OR, $\oplus$ es XOR. Los bloques que se arman por composición jerárquica de un mux2 o un sumador de 1 bit (barrel shifters, mux de B , mux de fuente del display) no llevan un mapa de Karnaugh de múltiples entradas propio: se documenta la tabla de la celda base y cómo se replica, en vez de un mapa de 6 entradas que no simplificaría nada y sería ilegible.

2.1 Multiplexor 2:1 (mux2)

sel	d_1	d_0	y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

		$d_1 d_0$			
		00	01	11	10
sel	0		1	1	
	1			1	1

$$y = \overline{sel} d_0 + sel d_1$$

SOP mínima, no simplificable más (dos implicantes primos esenciales, sin consenso útil). 4 compuertas: 1 NOT + 2 AND + 1 OR. Es el bloque más instanciado del diseño: aparece en swap, shifters, mux de B , mux de fuente del display, y en todos los registros con retención (FSM, contadores, **reg4**, debounce).

2.2 Sumador completo de 1 bit (full_adder)

a	b	c_{in}	s	c_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

		s				c_{out}			
		$b c_{in}$				$b c_{in}$			
		00	01	11	10	00	01	11	10
a	0		1		1			1	
	1	1		1			1	1	1

$$s = a \oplus b \oplus c_{in} \quad c_{out} = ab + a c_{in} + b c_{in} = ab + (a \oplus b) c_{in}$$

El mapa de s es un tablero de ajedrez (función XOR, sin implicantes de más de una celda). La forma implementada de c_{out} reutiliza $a \oplus b$, ya calculado para s : 2 XOR + 2 AND + 1 OR = 5 compuertas en vez de las 6 que saldrían de tomar los tres implicantes primos por separado.

2.3 Sumador de 4 bits (adder4)

Cuatro **full_adder** en cadena (ripple-carry): $s_i = a_i \oplus b_i \oplus c_i$, $c_{i+1} = a_i b_i + (a_i \oplus b_i) c_i$, $c_0 = c_{in}$. El acarreo c_4 se descarta, lo que da directamente la aritmética módulo 16 pedida. 20 compuertas (4×5). Verificado con las 512 combinaciones de a , b , c_{in} .

2.4 Acondicionador de resta

$A - B = A + \overline{B} + 1$ (módulo 16). Con una señal **sub**:

sub	y_i	y_i^c	$y_i^c = y_i \oplus \text{sub}, \quad c_{in} = \text{sub}$
0	0	0	
0	1	1	
1	0	1	
1	1	0	

Con **sub**=0 el operando pasa intacto y $c_{in} = 0$ (suma). Con **sub**=1 se invierte cada bit y el mismo **sub** entra como acarreo inicial, aportando el +1 del complemento a dos: 4 XOR, sin restador aparte.

2.5 Decodificador de operación (op_decoder)

op_2	op_1	op_0	sel_rst	sel_add	sel_sub	sel_rsub	sel_shl	sel_shr
0	0	0	1	0	0	0	0	0
0	0	1	0	1	0	0	0	0
0	1	0	0	0	1	0	0	0
0	1	1	0	0	0	1	0	0
1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	1
1	1	0	0	0	0	0	0	0
1	1	1	0	0	0	0	0	0

Tabla 6: 110 y 111 son códigos inválidos: las seis salidas quedan en 0.

Cada salida es exactamente un mintermino de tres literales:

$$\begin{aligned} \text{sel_rst} &= \overline{op_2} \overline{op_1} \overline{op_0} & \text{sel_add} &= \overline{op_2} \overline{op_1} op_0 & \text{sel_sub} &= \overline{op_2} op_1 \overline{op_0} \\ \text{sel_rsub} &= \overline{op_2} op_1 op_0 & \text{sel_shl} &= op_2 \overline{op_1} \overline{op_0} & \text{sel_shr} &= op_2 \overline{op_1} op_0 \end{aligned}$$

110 y 111 podrían tratarse como don't-care: por ejemplo sel_shl se reduciría a $op_2 \overline{op_0}$ (Figura 3). Se decidió **no aprovecharlos**: con las expresiones completas, un código inválido deja las seis líneas en 0 y la ALU entrega 0000, además de garantizar que el decodificador sea one-hot (lo que necesita el enmascarado de salida de la ALU, sección 3). El costo es una entrada más en dos de las seis compuertas AND.

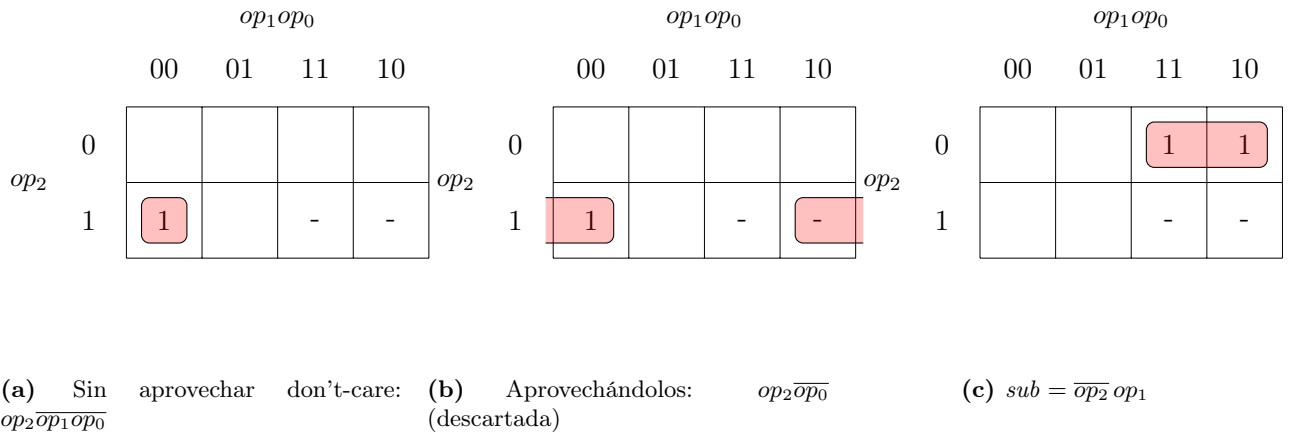


Figura 3: sel_shl con y sin don't-care, y la señal derivada **sub**.

Las señales de control de la ALU se derivan de las líneas ya decodificadas:

$$sub = sel_sub + sel_rsub = \overline{op_2} op_1, \quad swap = sel_rsub.$$

En el código, `sub` es un OR de dos líneas que ya existen (una compuerta), equivalente a la expresión mínima.

2.6 Intercambio de operandos y barrel shifters

Para $B - A$ se reutiliza la resta $x - y$ intercambiando las entradas con ocho `mux2`: $x_i = \overline{swap} a_i + swap b_i$, $y_i = \overline{swap} b_i + swap a_i$. Los shifters operan sobre A directo, no sobre x : el intercambio sólo afecta la vía aritmética.

Un desplazamiento de 0 a 3 posiciones se arma en dos etapas de `mux2` (desplaza 1 si sh_0 , desplaza 2 si sh_1), rellenando con 0 las posiciones que quedan vacías:

sh_0	t_3	t_2	t_1	t_0	sh_1	y_3	y_2	y_1	y_0	sh_0	t_3	t_2	t_1	t_0	sh_1	y_3	y_2	y_1	y_0
0	a_3	a_2	a_1	a_0	0	t_3	t_2	t_1	t_0	0	a_3	a_2	a_1	a_0	0	t_3	t_2	t_1	t_0
1	a_2	a_1	a_0	0	1	t_1	t_0	0	0	1	0	a_3	a_2	a_1	1	0	0	t_3	t_2

(a) Shift left

(b) Shift right

Tabla 7: Tablas por etapa de los barrel shifters ($sh = B[1:0]$).

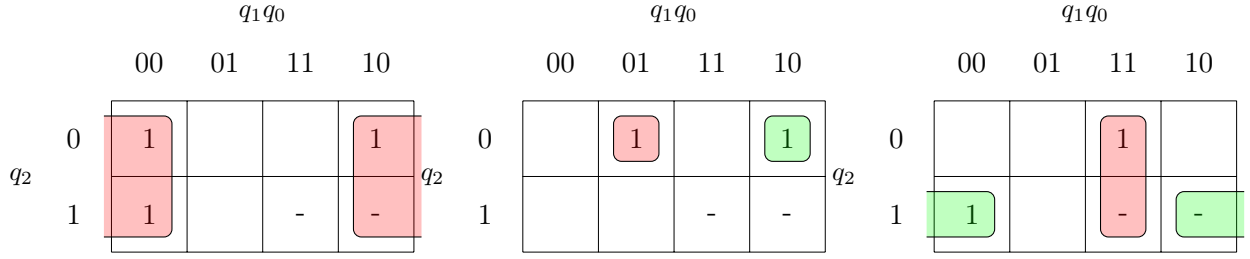
Los muxes con una entrada 0 se reducen a un AND ($\overline{sh} \cdot a$) en síntesis. 8 `mux2` por dirección = 32 compuertas cada uno.

2.7 Selector de operación: contador módulo 6 (`op_selector`)

El estado $q_2q_1q_0$ recorre $000 \rightarrow \dots \rightarrow 101 \rightarrow 000$ al subir y al revés al bajar; 110 y 111 son don't-care porque nunca se alcanzan.

Estado			Subir			Bajar		
q_2	q_1	q_0	n_2	n_1	n_0	n_2	n_1	n_0
0	0	0	0	0	1	1	0	1
0	0	1	0	1	0	0	0	0
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	0	1	0
1	0	0	1	0	1	0	1	1
1	0	1	0	0	0	1	0	0
1	1	0	—	—	—	—	—	—
1	1	1	—	—	—	—	—	—

Tabla 8: Próximo estado del selector de operación.



(a) n_0 (ambos sentidos) = $\overline{q_0}$

(b) n_1 subir = $\overline{q_2}(q_1 \oplus q_0)$

(c) n_2 subir = $q_1q_0 + q_2\overline{q_0}$

Figura 4: Mapas de Karnaugh del contador módulo 6 (subida). Bajada es análoga.

Expresiones mínimas (aprovechando los don't-care):

$$\text{subir: } n_0 = \overline{q_0}, \quad n_1 = \overline{q_2}(q_1 \oplus q_0), \quad n_2 = q_1q_0 + q_2\overline{q_0},$$

$$\text{bajar: } n_0 = \overline{q_0}, \quad n_1 = q_1q_0 + q_2\overline{q_0}, \quad n_2 = \overline{q_1}(q_2 \odot q_0).$$

Nota de consistencia. El código no usa exactamente estas formas mínimas para la bajada: implementa $n_1 = \overline{q_2}q_1q_0 + q_2\overline{q_1}\overline{q_0}$ y $n_2 = \overline{q_2}\overline{q_1}\overline{q_0} + q_2\overline{q_1}q_0$, la forma de minterminos completos (sin aprovechar los don't-care). Ambas formas son funcionalmente equivalentes en los seis estados alcanzables, verificado en `op_selector_tb`; se prefirió la forma completa por la misma razón que en el decodificador de operación: si el registro cayera en 110 o 111 por algún transitorio, esta forma lo devuelve a un estado válido en un ciclo.

2.8 Contador módulo 16 (counter4)

Aquí el módulo 16 es el desborde natural de 4 bits: no hace falta lógica de vuelta, sólo descartar el acarreo/préstamo de salida.

q_i	c_i	n_i	c_{i+1}	q_i	b_i	n_i	b_{i+1}
0	0	0	0	0	0	0	0
0	1	1	0	0	1	1	1
1	0	1	0	1	0	1	0
1	1	0	1	1	1	0	0

$$\text{subir: } n_i = q_i \oplus c_i, \quad c_{i+1} = c_i q_i, \quad c_0 = 1 \quad \text{bajar: } n_i = q_i \oplus b_i, \quad b_{i+1} = b_i \overline{q_i}, \quad b_0 = 1$$

Con $c_0 = b_0 = 1$, el bit 0 queda igual en los dos sentidos ($n_0 = \overline{q_0}$) y $c_1 = q_0$, $b_1 = \overline{q_0}$ salen sin compuertas adicionales: una sola señal se comparte. La misma cadena de acarreo (sólo la de subida, 15 bits) es el **prescaler**: $tick = c_{15} = q_{14}q_{13} \cdots q_0$, o sea el contador en todos unos, sin necesitar comparador.

2.9 Máquina de estados de control (fsm_control)

s_1	s_0	n_1	n_0	en_op	en_A	en_B	en_R
0	0	0	1	1	0	0	0
0	1	1	0	0	1	0	0
1	0	1	1	0	0	1	0
1	1	0	0	0	0	0	1

Tabla 9: Próximo estado (si *avanzar*=1) y decodificación one-hot.

Con esta codificación binaria, $s_0s_1s_2s_3 \rightarrow$ recorrer los cuatro estados es contar en binario: $n_1 = s_1 \oplus s_0$, $n_0 = \overline{s_0}$ (no requiere mapa de Karnaugh, sale directo de la tabla). Señales de control, todas combinacionales:

$$\begin{aligned} en_B &= s_1 \overline{s_0} \text{ (detecta S2)} & sel_op2 &= usar_anterior \cdot en_B \\ load_R &= en_B \cdot (confirmar + usar_anterior) & avanzar &= confirmar + sel_op2 \end{aligned}$$

La retención (sin pulso, el estado se mantiene) usa un mux2 por bit: $d_1 = avanzar n_1 + \overline{avanzar} s_1$, análogo para d_0 .

2.10 Signo y valor absoluto (abs4)

$signo = V_3$; $|V| = (V \oplus V_3) + V_3$ (el bit 3 del XOR es $V_3 \oplus V_3 = 0$ siempre, así que son 3 XOR, no 4, más una instancia de **adder4** con $c_{in} = V_3$).

$V_3V_2V_1V_0$	valor	signo	$D_3D_2D_1D_0$	$ V $
0000	0	0	0000	0
0001	1	0	0001	1
0010	2	0	0010	2
0011	3	0	0011	3
0100	4	0	0100	4
0101	5	0	0101	5
0110	6	0	0110	6
0111	7	0	0111	7
1000	-8	1	1000	8
1001	-7	1	0111	7
1010	-6	1	0110	6
1011	-5	1	0101	5
1100	-4	1	0100	4
1101	-3	1	0011	3
1110	-2	1	0010	2
1111	-1	1	0001	1

Tabla 10: Tabla de **abs4**. -8 es el único valor cuyo patrón de bits no cambia (no existe +8 en el rango). D nunca supera 8: 9-15 son don't-care para el decodificador de 7 segmentos.

2.11 Decodificador de 7 segmentos (seg7_decoder)

Entrada $D_3D_2D_1D_0$ (magnitud 0-8), salidas $a..g$ en lógica positiva. 9-15 son don't-care y se aprovechan en los siete mapas.

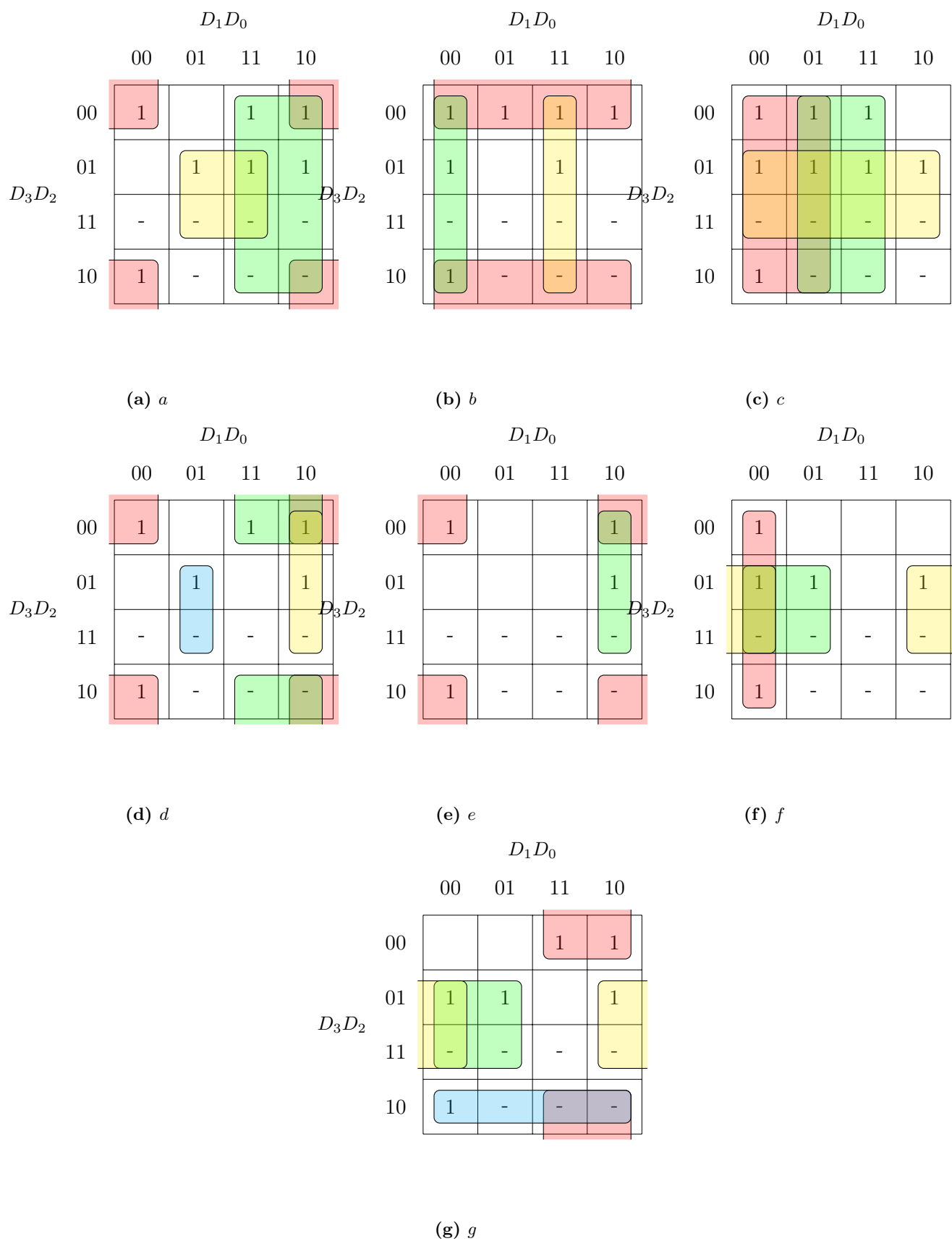


Figura 5: Mapas de Karnaugh de los siete segmentos.

$$\begin{aligned}
a &= \overline{D_2 D_0} + D_1 + D_2 D_0 & b &= \overline{D_2} + \overline{D_1 D_0} + D_1 D_0 & c &= \overline{D_1} + D_0 + D_2 \\
d &= \overline{D_2 D_0} + \overline{D_2 D_1} + D_1 \overline{D_0} + D_2 \overline{D_1} D_0 & e &= \overline{D_0} (\overline{D_2} + D_1) & f &= \overline{D_1 D_0} + D_2 \overline{D_1} + D_2 \overline{D_0} \\
g &= \overline{D_2 D_1} + D_2 \overline{D_1} + D_2 \overline{D_0} + D_3
\end{aligned}$$

D_3 aparece sólo en g : el 8 enciende los mismos segmentos $a..f$ que el 0, así que esos seis mapas agrupan ambos y eliminan D_3 ; en g no se puede porque el 0 tiene $g = 0$ y el 8 tiene $g = 1$. Cinco productos se comparten entre segmentos y se calculan una sola vez: $\overline{D_2 D_0}$ (a, d), $\overline{D_1 D_0}$ (b, f), $\overline{D_2 D_1}$ (d, g), $D_2 \overline{D_1}$ (f, g), $D_2 \overline{D_0}$ (f, g). Con eso el decodificador completo queda en 3 NOT + 10 AND + 7 OR = 20 compuertas.

El display izquierdo sólo usa el segmento $g = \text{signo}$ ($a..f = 0$, conexión directa). En S0, `dash_s0` fuerza un guion en ambos: $g'_{izq} = g_{izq} + en_op$, $x'_{der} = x_{der} \cdot \overline{en_op}$ (para $x \in a..f$), $g'_{der} = g_{der} + en_op$.

3 Implementación de las operaciones mediante compuertas

Cada expresión de la sección anterior se tradujo directo a primitivas de Verilog: un término producto es un `and`, una suma es un `or`, un literal negado es un `not` compartido y un $\oplus$ es un `xor`. Los fragmentos de código de esta sección son los módulos reales de `src/`.

3.1 Celdas básicas

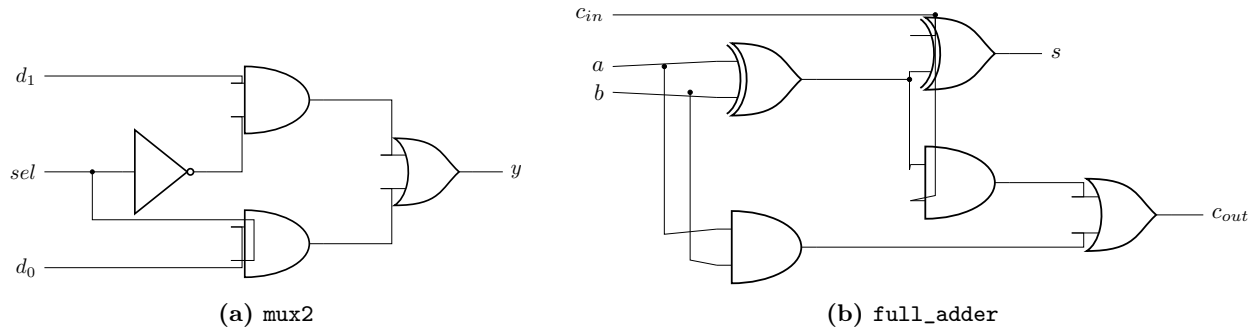


Figura 6: Celdas básicas de la jerarquía.

```
// mux2.v
not n0 (n0, sel);
and a0 (t0, d0, n0);
and a1 (t1, d1, sel);
or o0 (y, t0, t1);
```

```
// full_adder.v
xor x0 (ab, a, b);
xor x1 (s, ab, cin);
and a0 (c_ab, a, b);
and a1 (c_abcin, ab, cin);
or o0 (cout, c_ab, c_abcin);
```

3.2 Sumador de 4 bits

```
// adder4.v
full_adder fa0 (a[0], b[0], cin, sum[0], c1);
full_adder fa1 (a[1], b[1], c1, sum[1], c2);
full_adder fa2 (a[2], b[2], c2, sum[2], c3);
full_adder fa3 (a[3], b[3], c3, sum[3], cout); // cout se descarta en la ALU
```

3.3 Decodificador de operación

```
// op_decoder.v
not v2 (n2, op[2]); not v1 (n1, op[1]); not v0 (n0, op[0]);
and d0 (sel_rst, n2, n1, n0); // 000
and d1 (sel_add, n2, n1, op[0]); // 001
and d2 (sel_sub, n2, op[1], n0); // 010
and d3 (sel_rsub, n2, op[1], op[0]); // 011
and d4 (sel_shl, op[2], n1, n0); // 100
and d5 (sel_shr, op[2], n1, op[0]); // 101
```

3.4 ALU

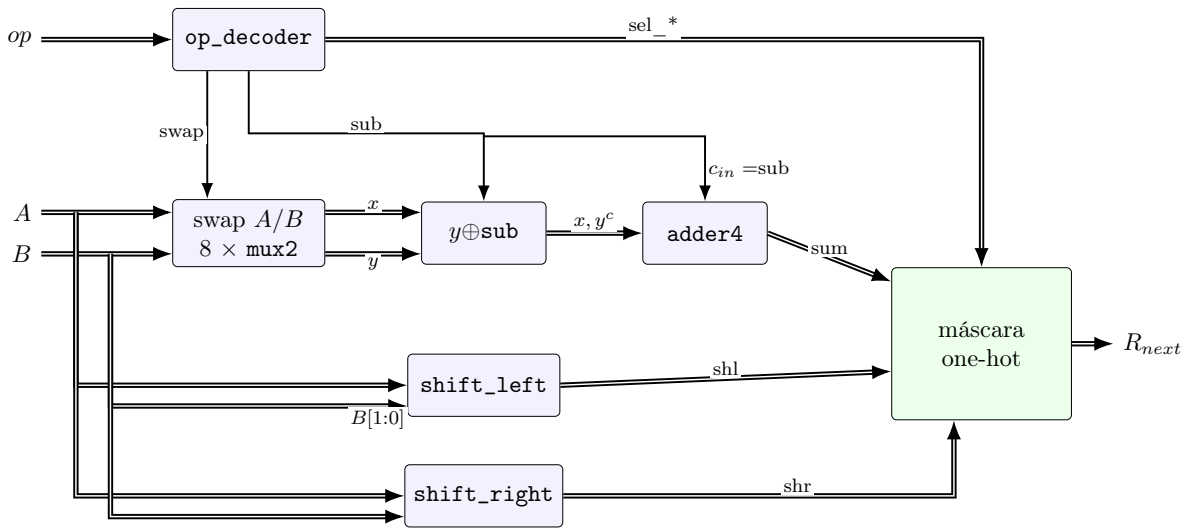


Figura 7: Estructura de la ALU (alu.v).

```
// alu.v (extracto)
or c0 (sub, sel_sub, sel_rsub);
buf c1 (swap, sel_rsub);
mux2 sx0 (a[0], b[0], swap, x[0]); // ... bits 1..3
mux2 sy0 (b[0], a[0], swap, y[0]); // ... bits 1..3
xor k0 (yc[0], y[0], sub); // ... bits 1..3
adder4 add (x, yc, sub, sum, cout); // cout descartado -> modulo 16
shift_left sl (a, b[1:0], shl);
shift_right sr (a, b[1:0], shr);
and ga0 (m_add[0], sum[0], sel_add); and gs0 (m_sub[0], sum[0], sel_sub);
and gr0 (m_rsub[0], sum[0], sel_rsub); and gl0 (m_shl[0], shl[0], sel_shl);
and gh0 (m_shr[0], shr[0], sel_shr); // ... bits 1..3
or f0 (r[0], m_add[0], m_sub[0], m_rsub[0], m_shl[0], m_shr[0]);
```

3.5 Barrel shifter

```
// shift_left.v
mux2 p3 (a[3], a[2], sh[0], t[3]); mux2 q3 (t[3], t[1], sh[1], y[3]);
mux2 p2 (a[2], a[1], sh[0], t[2]); mux2 q2 (t[2], t[0], sh[1], y[2]);
mux2 p1 (a[1], a[0], sh[0], t[1]); mux2 q1 (t[1], 1'b0, sh[1], y[1]);
mux2 p0 (a[0], 1'b0, sh[0], t[0]); mux2 q0 (t[0], 1'b0, sh[1], y[0]);
```

3.6 Contador y máquina de estados

```
// counter4.v (cadena de subida, bit 1 como ejemplo)
xor ib (n_inc[1], q[1], q[0]); and ic (c2, q[0], q[1]); // c[1] = q[0]
and s0 (inc_ok, en, inc); not s1 (ninc, inc); and s2 (dec_raw, dec, ninc);
and s3 (dec_ok, en, dec_raw); nor s4 (hold, inc_ok, dec_ok);
and g10 (m_inc[1], n_inc[1], inc_ok); and g11 (m_dec[1], n_dec[1], dec_ok);
and g12 (m_hold[1], q[1], hold); or g13 (m_or[1], m_inc[1], m_dec[1], m_hold[1]);
and r2 (d[1], m_or[1], nrst);
always @(posedge clk) q <= d; // unico bloque secuencial
```

```
// fsm_control.v
not v1 (ns1, q1); not v0 (ns0, q0);
and e0 (en_op, ns1, ns0); and e1 (en_A, ns1, q0); and e2 (en_B, q1, ns0); and e3 (en_R
, q1, q0);
and c0 (sel_op2, usar_anterior, en_B); or c1 (any_conf, confirmar, usar_anterior);
and c2 (load_R, en_B, any_conf); or c3 (avanzar, confirmar, sel_op2);
xor p1 (n1, q1, q0); buf p0 (n0, ns0);
mux2 h1 (q1, n1, avanzar, m1); mux2 h0 (q0, n0, avanzar, m0);
and r1 (d1, m1, nrst); and r2 (d0, m0, nrst);
```

3.7 Valor absoluto y 7 segmentos

```
// abs4.v
xor x0 (x[0], v[0], v[3]);
xor x1 (x[1], v[1], v[3]);
xor x2 (x[2], v[2], v[3]);
adder4 inc ({1'b0, x[2:0]}, 4'b0000, v[3], d, cout);
```

```
// seg7_decoder.v (productos compartidos y dos salidas de ejemplo)
and q0 (p_n2n0, n2, n0); and q3 (p_d2n1, D2, n1); and q4 (p_d2n0, D2, n0);
or sa (seg[6], p_n2n0, D1, p_d2d0); // a
or q9 (e_or, n2, D1); and se (seg[2], n0, e_or); // e
or sg (seg[0], p_n2d1, p_d2n1, p_d2n0, D3); // g
```

3.8 Conteo de compuertas por módulo

Cada módulo documenta su propio conteo en el encabezado del archivo:

Módulo	Compuertas	FF	Detalle
mux2	4	–	1 NOT + 2 AND + 1 OR
full_adder	5	–	2 XOR + 2 AND + 1 OR
adder4	20	–	4 × full_adder
shift_left/shift_right	32 c/u	–	8 × mux2
op_decoder	9	–	3 NOT + 6 AND ₃
abs4	–	–	3 XOR + adder4 (20)
seg7_decoder	20	–	3 NOT + 10 AND + 7 OR
reg4	21	4	4 × mux2 + 4 AND reset + 1 NOT
reg4_ini	16	4	4 × mux2, sin reset
dash_s0	9	–	6 AND + 2 OR + 1 NOT
por	–	4	4 buf + 1 NOT
fsm_control	24	2	2 NOT + 2 mux2 + ~10 compuertas de control
debounce	17	4	3 mux2 + AND ₃ + OR ₃ + AND + OR + NOT
edge_detect	–	1	NOT + AND
boton	20	5	debounce + edge_detect
prescaler	44	15	1 NOT + 14 XOR + 14 AND + 15 AND reset

Tabla 11: Conteo de compuertas primitivas propias de cada módulo (según su propio encabezado en el código); no incluye los submódulos que instancia. `alu`, `counter4` y `op_selector` se describen por estructura en las secciones anteriores en vez de un total único.

El resultado de síntesis real (174 celdas lógicas del iCE40 HX1K, sección 1.7) es la cifra que importa para la FPGA: la diferencia con la suma de compuertas primitivas se debe a que cada SB_LUT4 del dispositivo absorbe varias compuertas al mapear la netlist.